A13｜把输入整理成一份可靠报告

这是 A 册的综合连接：**文件 → 原始记录 → 有效工单或拒绝原因 → 完整报告 → 命令行结果**。重点是交接，不是再背一遍前面的语法名称。

一张工单怎样穿过程序？

当前问题	关系或短用法	记到什么程度
文件内容怎样进入程序？	`load_rows` 按扩展名选 CSV / JSON 解析，返回原始记录列表；CSV 的 `"1"` 与 JSON 的 `1` 此时可以类型不同。	**解析成功不等于记录符合业务规则。** 类型提示也不执行输入检查。
谁决定这条能不能用？	`validate_row` 检查类型、清理值、检查范围，交回 `(ticket, None)` 或 `(None, issue)`。	能读懂两个返回位置怎样让调用者分路，不靠有没有打印错误猜结果。
多条记录的状态谁保存？	`organize_rows` 保存 `accepted`、`issues`、`seen_ids`，处理重复、排序和统计。	有效列表保存完整记录，集合只记已接受 ID。**只有接受过的记录占用编号。**
内存报告何时成为磁盘文件？	`write_report` 先写同目录临时文件，关闭后再替换目标报告。	**旧目标保护来自顺序，不是单靠 except。** Path 细节查用；本例同一输出只安排一个写入进程。
哪层把结果交回命令行？	`run` 组织调用并返回整数；直接运行入口用 `sys.exit(run(sys.argv))` 转成退出码。	返回值、打印文字、报告各有接收者；测试导入模块不自动启动命令行入口。

这些函数名是本项目的查找入口，不是 Python 固定语法。回看「函数怎样接过上一段的结果」「文件内容先成为原始记录」「完整报告怎样才成为磁盘上的新目标」。

这里的 CSV 路线假定表头不重复、每条记录的字段数与表头一致；当前程序没有完整检查这两项。`strict=True` 只加强 CSV 语法解析，不会补出上述结构校验。接收外部 CSV 时，回 A08 的字段检查与本章「文件内容先成为原始记录」核对，不能因报告生成就宣布任何 CSV 都已经过完整验证。

为什么不能只靠转换和容器完成规则?

当前问题	关系或短用法	记到什么程度
int 转得出来，就能接受吗?	`"urgent"` 转换失败；`"9"` 转成整数后仍超出 1–3；`True` 虽可转成 1，本程序仍拒绝 bool。	**原始类型、转换、业务范围是不同判断。** 普通业务拒绝不都等于异常。
空负责人怎样处理?	owner 缺少、为 `null` 或空白文字时统一为 `unassigned`，提供其他类型仍拒绝。	不能把数字 `0` 或空列表当合法“未分配”。id、subject 清理后不能为空，status 仅允许 open / pending / closed。具体规则查项目。
建立集合就会自动拒绝重复吗?	检查 ID 是否在 `seen_ids`；重复则记录 issue 并 `continue`，不再加入有效列表。	容器保存事实，**判断与控制流执行规则**；先前没通过验证的记录不占 ID。
怎样排序、怎样计数?	先按整数 priority，再按文字 id 排序；统计只针对 accepted。	同优先级 ID 按文字比较，不按尾部数字。输入条数 = 接受条数 + 拒绝条数；一条 issue 可有多个原因。

回看：「一条记录怎样取得继续使用的资格」「多条记录怎样变成一份报告」。

T-201 的优先级 `1` 变成标签 `high`；T-202 的空 owner 变成 `unassigned`。两条都有效，报告接受 2、拒绝 0。混合 CSV 后四条分别因转换、空标题、重复 ID、非法状态被拒绝，仍写出报告：接受 3、拒绝 4。回看「先看 T-201 最后到了哪里」「坏记录怎样进入报告，而让下一条继续」。这些值帮助核对随书样例，不是要背的业务常识。

哪里失败，决定留下什么结果

当前问题	关系或短用法	记到什么程度
本程序的退出码怎么读?	`0` 全部有效且报告已写；`1` 参数数量不对；`2` 报告已写但有拒绝；`3` 输入 / 格式约定不符；`4` 权限、编码或其他文件操作失败。	**`2` 不等于报告没生成。** 编号查本程序约定，不当 Python 通用表背。
输入坏了与写出失败有何不同?	坏 JSON 在写出前停止；写出失败在写入阶段处理。异常类型和发生位置一起决定外部含义。	沿阶段找最早断点；临时文件策略不保证断电、强杀或并发写入都能恢复。
测试能证明到哪?	业务测试比清理后值；跨格式测试比整理后工单；受控失败测试查退出、诊断与旧目标。	**断言查了什么，才支持什么。** 替换整个写入函数，不能单独证明真实写入函数的清理行为。
改规则后，怎样不让测试迁就代码?	先定新预期，加能区分新旧规则的测试，再改对应函数并重跑。	从差异追回交接点；Git 查清变化，A12 针对新提交重新验证重建。

回看：「最外层怎样告诉命令行这次发生了什么」「先用简单测试固定业务，再检查失败边界」「用一次小改动检查自己能否找回关系」。`tmp_path`、`capsys`、`monkeypatch` 分别帮助隔离临时文件、收集输出、受控替换；完整测试写法在对应例子查，不默写整套。

运行前按「在自己的实验目录里运行一次」复制完整项目，不在随书参考副本内产生状态。输入和输出用分开的普通文件，核对输出将替换谁，不把任意旧文件当可丢弃报告。

本章把整册接成：**解析器解释格式，验证函数判断一条，整理函数保存跨记录状态，写出函数改变磁盘，入口向外说明结果**。当前程序仍是本地离线工具；B 册才加入 HTTP 协作，不要把模块导入画成网络请求。